

PROJECT AND TEAM INFORMATION

Project Title

MediSense AI – A Hybrid, Self-Built k-NN Based Clinical Decision Support and Triage System

Student / Team Information

Team Name: Team #	Codeblue
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	Verma, Krish – 2510370349 2510370349@geu.ac.in 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	Kumar Kanu, shivam – 2510018717 2510018717@geu.ac.in 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	Garg, Suhani – 2510350042 2510350042@geu.ac.in 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	Sumit – 2510370550 2510370550@geu.ac.in 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Across healthcare providers of every size, patient intake still runs on guesswork rather than data. In small and mid-sized clinics, there is usually no decision-support tool at all: routing decisions depend on whichever staff member happens to be on duty, a genuine emergency can end up waiting behind routine walk-ins simply because there is no automated priority mechanism, and every patient is effectively treated as a first-time case even though similar cases have almost certainly occurred before. Doctor availability and patient routing are handled ad hoc, by memory or a phone call, and medicine shortages tend to surface only at the point of prescription, when it is already too late to plan around them.

Large hospitals face a related but very different version of the same problem. here the issue is scale rather than absence of process: patient volumes overwhelm manual triage judgment across departments, coordinating dozens of doctors and specialties by hand becomes error-prone, and emergency cases must be prioritised automatically and consistently rather than depending on a busy front desk noticing them in time. hospitals also sit on tens of thousands of historical records that are simply stored and never used, even though a dataset of that size is exactly what makes a technique like k-Nearest Neighbours statistically meaningful. naive linear search and unsorted queues, at that volume, become genuine performance bottlenecks rather than academic inefficiencies.

Solving this matters because triage delays and uncoordinated routing directly affect patient outcomes, and because a clinic or hospital that cannot learn from its own historical data is discarding a resource it already has. this project treats that as a data structures problem as much as a healthcare one: the right hash tables, queues, and search structures turn stored records into a working decision-support system instead of a passive archive.

State of the Art / Current solution (1 pt)

Most small and mid-sized clinics today rely entirely on staff experience and paper or basic digital registers; there is no predictive or decision-support layer, and triage priority is judged informally at the front desk. larger hospitals typically run Hospital Management Systems or Electronic Health Record platforms, but these are largely record-keeping and scheduling tools – they store patient, doctor, and appointment data reliably, yet most do not actively use that stored history to help route a new patient or suggest a likely diagnosis. where predictive tools do exist, they are usually built on heavyweight external machine-learning libraries, which makes them opaque to the people using them and unsuited to a course that is specifically about implementing data structures and OOP concepts from first principles. As a result, the gap is not a lack of digital systems – it is the absence of a lightweight, transparent, syllabus-native decision-support layer sitting on top of the data hospitals and clinics already collect. MediSense AI is designed to fill exactly that gap, using hand-built structures rather than an imported ml stack

Project Goals and Milestones (2 pts)

Goals

1. Build a working clinical decision-support and triage system in C that maps every Data Structures topic (hashing, sorting, queues, linked lists, sparse matrices, dynamic memory, file structures) onto a real, functioning module rather than an isolated exercise.
2. Simulate the full OOP (C++) syllabus – classes, encapsulation, inheritance, polymorphism, operator overloading, exception handling – using equivalent C constructs, so one codebase genuinely satisfies both courses.
3. Hand-build a lightweight k-Nearest Neighbours engine from scratch, so historical case data actively improves future predictions instead of sitting unused.
4. Support three distinct user roles – Doctor, Patient, and Receptionist – each restricted to the tools their job actually needs.

Milestones

1. Core data layer: patient, doctor and case-record structs; dynamic arrays; file persistence for patients, doctors, and case history.
2. Symptom hashing: hash table for symptom-to-disease lookup, built over a sparse disease-symptom matrix; collision handling.
3. k-NN engine: KNN_addCase() / KNN_predict(), Euclidean distance computation, custom Quick Sort on distances, majority vote over the top-k neighbours.
4. Triage logic: priority queue for emergency cases, FIFO queue for routine bookings, linked-list visit history per patient.
5. OOP mapping & roles: opaque-struct encapsulation, inheritance (Person → Doctor), function-pointer polymorphism for doctor recommendations, and the three-role login system.
6. Integration, testing & demo: end-to-end walkthrough on sample patient data, report and presentation.

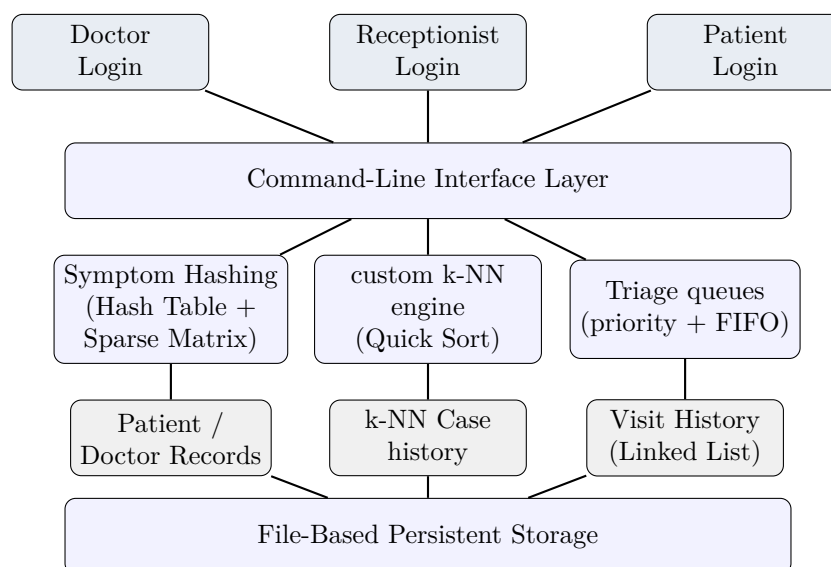
Project Approach (3 pts)

MediSense AI is implemented entirely in C, deliberately, so that every C++ OOP concept required by the syllabus has to be recreated by hand using structs and function pointers – the same way STL containers or classes work under the hood. A patient's intake runs through two stages. stage 1 hashes the reported symptoms against a disease-symptom hash table built on top of a sparse disease $\times$ symptom matrix, producing a percentage symptom match per candidate disease. stage 2 feeds the patient's normalised vitals into our own k-NN engine: distances to every stored historical case are computed, sorted with a custom Quick Sort implementation, and the five nearest neighbours vote on a diagnosis. the two signals are combined into a single weighted match score that drives doctor recommendation by specialty.

On the OOP side, we lean on constructs that mirror C++ directly: an opaque struct plus a small set of accessor functions `KNN_addCase()`, `KNN_predict()` stands in for encapsulation, since the rest of the program never touches the underlying array or sort logic; a base `Person` struct embedded as the first member of a `Doctor` struct gives us inheritance; a function pointer stored per doctor instance acts as a hand-built vtable for polymorphic recommendation behaviour; named functions such as `Patient_equals()` stand in for operator overloading; and `setjmp/longjmp` or error-code enums handle exception-style vitals validation.

Triage itself is handled with a priority queue for flagged emergencies and a plain FIFO queue for routine bookings, so urgent patients are never stuck behind non-urgent walk-ins. Each visit is appended to the patient's medical history as a new linked-list node, and every doctor-confirmed diagnosis is appended back into the k-NN dataset, so the system's predictions keep improving as more real cases are logged. Data is persisted to flat files, and the whole system is exposed through three separate command-line logins: Doctor, Patient, and Receptionist.

System Architecture (High Level Diagram)(2 pts)



Three role-based logins (Doctor, Receptionist, Patient) sit on top of a shared interface layer. Requests flow into three core engines – symptom hashing, the custom k-NN predictor, and the triage queues – each backed by its own on-disk store, all persisted through a common file-based storage layer

Project Outcome / Deliverables (1 pts)

The project will deliver a working C application implementing the full MediSense AI pipeline: symptom-hash based disease matching, a hand-built k-NN engine for historical case comparison, priority-based emergency triage alongside FIFO routine booking, and file-backed persistence for patients, doctors, and case history. It will expose three functioning role-based logins – Doctor, Patient, and Receptionist – each scoped to the actions relevant to that role. Alongside the source code, we will deliver this written proposal/report, a system architecture diagram, a worked end-to-end example demonstrating the prediction pipeline on sample data, a mapping table showing exactly where each Data Structures and OOP syllabus topic is used in the codebase, and a final presentation and live demo. a short project readme documenting how to build and run the system, along with sample datasets used for testing the k-NN engine, will also be included

Assumptions

We assume the symptoms and vitals entered at intake are reasonably accurate and complete, since the prediction quality depends directly on input quality. we assume a starting historical dataset of roughly 40 confirmed cases for development and testing, growing over time as new diagnoses are logged, and that $k = 5$ nearest neighbours is a reasonable default for a dataset of this size. we assume single-machine, file-based storage is sufficient for this academic prototype rather than a networked database, and that the system is a decision-support aid for staff – not a replacement for a doctor's judgement or professional medical advice.

References

1. Course lecture notes and lab material – Data Structures (C) and Object-Oriented Programming (C++).
2. A. A. et al. (2025). “A dynamic model using k-NN algorithm for predicting diabetes and breast cancer.” *Computers in Biology and Medicine*, 192, 110276. DOI: 10.1016/j.combiomed.2025.110276.
3. J. M. Ruiz Mejia and D. B. Rawat, “Exploring the Advancements of AI Enabled Clinical Decision Support Systems for Patient Triage in Healthcare,” in *Proc. IEEE Int. Conf. E-Health Networking, Application & Services (HealthCom)*, Japan, 2024, pp. 1–4, doi: 10.1109/HealthCom60970.2024.10880833
4. T. Thangalakshmi, N. Mohankumar, L. N. Jayanthi, Y. M. Blessy, R. Meenakshi, and M. Rajmohan, “Telemedicine Platforms with Cloud-Based AI-Driven Diagnostics in Hospitals,” in *Proc. 2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*, India, 2025, pp. 137–143, doi: 10.1109/ICVADV63329.2025.10961133
5. Course syllabus documents used to build the topic-mapping table in Section 3 of the content reference.